
output: pdf_document: default html_document: default

Lecture 1 — Speaker Transcript

From Biological Question to Machine Learning Problem

How to read this script. This is continuous spoken prose — you can read it more or less verbatim. Slide markers and rough timings are in brackets at each section head. When there's a question to the room, the line you *ask* is in plain text, and the answer/steer you're guiding them toward follows on its own line, marked **A** (in red). Total target $\approx$ 90 minutes.

Slide 1 — Title

(~1 min)

Hi everyone, for those that do not know me, I am Zhan and I am the head of the data science unit at IEO. Within our group, we use a lot of computational tools to solve or try to answer questions.

The course seems long but actually it's not as there will be a lot of practice.

As you guys might have noticed, I changed the title of the course. The reason is that I wanna try something different. Instead of the standard course that you find also on youtube on machine learning, which explains the different type of models and the mathematics behind, my idea is really to give you some rules, some indication on how to approach a problem or a question using machine learning. So appreciate if you give feedbacks.

And the question or problem we wanna answer is the following: Can a tumour's transcriptome tell us who will recur — and who is safe?

The first part of the question is the ingredients we are gonna use, while the second part is the real question which we matter a lot.

Right now, somewhere, an oncologist is discussing with a woman with early breast cancer, trying to decide whether to give her chemo or not. Chemotherapy is toxic, and treating a patient that would not need it could cause real harm such as toxicity, cost for nothing. If we do not treat her, and the cancer comes back, she might have missed her shot at stopping a metastatic recurrence. That's a lifelong decision.

So again, this whole course is built around is a single question: **can the tumour's own gene expression — its transcriptome — help us to make that call better?** And just as important: why have so many attempts to do this failed?

Ask the room: Quick show of hands — who here has built a classifier before?

Slide 2 — Where this lecture sits

(~2 min)

So today we will not talk about modelling. Just concepts. So no algorithms. No logistic regression, no random forests, no neural nets.

Most ML courses throw a model at you on day one. We with the question and the data — because in genomics or transcriptomics, the model is almost never what makes the project successful or breaks it. Of course we will talk about the models in the coming days.

If you take one thing away today, make it this: in our field, the model is rarely the hard part. The hard part is everything *around* the model.

This is Section 1 of five. Today we frame the problem. Then we build a baseline, then feature engineering, then validation, and finally how to make an end-to-end ML that will not degrade in the clinical validation in terms of performance.

By the end of the course the goal is really to give you the tools to take a vague biological or clinical question into a well-posed prediction problem or machine learning problem so that your prediction tool will not sink in the validation phase.

Slide 3 — Learning objectives

(~2 min)

These are the objectives of today: since the problem or the question depends on the context, it's really important to understand the context. For this course, the question is a clinical question so we will describe the clinical context and the problem

Then we will explain how transcriptomic data gets represented for ML; How to define pick out features and define labels; We will try to understand why this data is difficult, namely that we have way more features compared to data points, we have batch effects in data generation, and the leakage can be hidden everywhere; Coming back to the number of features and patients, we will explain the curse of dimensionality without the maths; Try to explain why most of the published biomarkers fall apart at validation; and finally design a train/validation/test split that actually holds up in the external validation after publication

Watch the last two especially — **leakage and splitting**. Those are the ones that separate work that gets published but is wrong, from work that actually survives.

Slide 4 — The clinical setting

(~4 min)

Let's ground this to the clinical context. The data you guys are going to use are breast cancer transcriptomics data. In particular, a specific type of breast cancer that is characterise by Hormone Receptor positive (HR-positive), and HER2-negative breast cancer This is the most common subtype, about seventy percent of cases. Good news: it generally has a good prognosis, but there is

heterogeneity: some tumours need chemotherapy to be contained, some others not.

Now look at the two ways we can be wrong, because both hurt someone: On one side we have **Overtreatment**. Chemo carries heart damage, infertility, a risk of secondary cancers. It can give a huge hit to quality of life. So give it to someone who'd have been fine, and you've done real harm. On the other hand **Undertreatment**. The risk is a metastatic recurrence — a cancer that comes back and it's much harder to be cured anymore.

So a good biomarker here is not just finding the high-risk patients. It's but also about who we can safely say that they do not need more treatment.

Here's the catch. Imagine two patients who look identical on paper: same tumour size, same grade, same nodal status. But under the hood, their tumour biology can be completely different, and one relapses while the other never does. Clinical features alone can't tell them apart. That gap — the stuff we can't see by eye — is the whole reason we go looking at the transcriptome.

Ask the room: If our biomarker is going to be wrong sometimes, which mistake is worse — telling a patient who needs chemo to skip it, or telling a patient who could safely skip it that she needs it? Have a think. There's no single right answer — but depend on which one you pick, should change the kind of model you'd want or more precisely the metric you wanna use to evaluate the model performance.

And drop one intuition right now: that more treatment is automatically safer. In this group, more treatment can mean net harm.

Slide 5 — The biological question vs the ML question

(~4 min)

Here we have the main take away of the lecture. When a clinician or a biologist ask a question, this does not immediately correspond to a machine learning question that makes sense.

For instance The clinician's question is: "*Will this patient's tumour respond to treatment?*" This makes sense from a clinical point of view. It's exactly what you wanna answer. But it is **not** a well made machine learning problem. Not yet. It's too vague.

It only becomes an ML problem once we lock in four choices:

1. **What** exactly do we measure? (*the input*)
2. **What** are we predicting, and how precisely is it defined? (*the output*)
3. **When** in the patient's timeline do we measure?
4. **On whom** — which patients are even allowed into the study?

Every one of those is a *scientific* decision. It's something we decide, and then the algorithm would just execute. Everything downstream depends on this. If we made wrong decisions, the model won't know it and can't rescue it

Very important point is that the dataset does not necessary define the problem. The way we do it is that, we (together with the clinician) define the problem, and *then* goes looking for — or generates

— data that fits. If the data is not there, and there is no way to generate those that because it's too expensive, then we try to accomodate the problem wit the data we can generate.

Slide 6 — Defining the prediction target

(~5 min)

One of the most important and difficult thing in our field, which is oncology, is having a well defined and clinically interesting prediction target. In fact, when we say "predict response". What do we mean for response? What is the label?

One possibility is **pCR (pathologic complete response)**: no leftover invasive disease at surgery after neoadjuvant therapy. It's a tempting choice because you measure it early — *but* in luminal HR+/HER- disease it only weakly tracks survival (much more weakly than in triple-negative or HER2-positive).

Another possibility is the clinical benefit. This is great, it tells you who is gonna benefit from the treatment. Since we want to know who will benefit, we need an untreated arm which only clinical trials have in general. The third option is

- **Relapse-free survival**: closest to what patients actually care about — but it takes years, and brings censoring.

Now, in our practical session, we will be using exactly RFS, the relapse free survival. Our cohort we chose is the **METABRIC** — it is *not* a neoadjuvant trial, so it has no pCR at all, but we do not care. What it does have is long-term relapse and survival follow-up which is exactly our end point, the target. So in the practical, we'll predict **recurrence** — a relapse-based yes/no endpoint.

One important thing to say here is that the RFS is a time to event data, it is a combination of whether it an event has happened and when it happened. This comes with censoring. For our practical, we will make it simpler. We will binarise it by a time horizon threshold, but it costs us something. Meaning that patients for which we can't really define the class, for instance patients censored before the threshold, we will discard them. The second thing is that two patients that relapsed way far apart but after the threshold, they end up the in same category. We will come back to this in the latest session on friday

Slide 7 — Where the data come from

(~4 min)

Now where exactly does the data come from and exactly it's generated.

The workflow is the following: **biopsy** → **RNA extraction** → **array hybridisation** → **scanning** → **normalisation**. Each of these contains possible artifacts which one have to correct. For instance for the biopsy, how much is the tumor fraction, which part of the tumor do we get the biopsy, how was it conserved, was it fresh frozen which makes the RNA less degraded or was it FFPE, so formalin fixed? Each of these introduces counfondings. and it's all dependent

For instance the RNA extraction depends on how the biopsy was conserved and how much material

we have.

- *Biopsy*: which bit of the tumour? How much is actually tumour vs surrounding stroma?
- *Extraction*: RNA degrades, especially in old archived FFPE samples.
- Every later step is a chance to sneak a technical artefact into what you'll later treat as biological signal.

We're using **microarrays** in this course. They've mostly been replaced by RNA-seq, but they're still the foundation for a lot of the field's landmark, fully-annotated public cohorts — which is exactly why they're great for teaching.

METABRIC was run on Illumina; the second cohort, Loi GSE6532, on Affymetrix. Two different chemistries, measuring the same underlying biology. This is an example of batch effect or counfunding effects that can dominate your results.

Slide 8 — The transcriptomic data matrix

(~4 min)

The microarray or RNAseq produces the input data we are gonna use for our practical session, which is the transcriptomics matrix. In machine learning convention, **the rows corresponds to samples** (patients/tumours): one row is everything we measured about patient seventeen; **the columns = genes**: one column is how a single gene changes across all patients.

A typical study is a few hundred samples by around twenty thousand genes. In the metabric, you are lucky because you have more than 1000 samples.

Remember this: the row as sample is what machine learning expects. What scikitlearn expects. But the tools in bioinformatics give you actually the opposite! If you run nfcore/sarek on multiple samples, rows are genes. So, very important. remember to transpose. if you do not, we will be developing something that is not what you want! and actually this is a common problem

Double check always before doing anything in your exploratory phase that rows are samples!

Slide 9 — Features and labels, concretely

(~3 min)

Given the expression matrix, the features are the genes. For each patients we have 20k features. The label is what we defined few slides back

- **y (label)**: the single response column we spent so long defining on Slide 6.

You can also add clinical variables into X (age, tumour size, grade, nodal status). And that raises a question we'll keep circling back to: **does the transcriptome actually add anything on top of those cheap, already-available clinical variables?** A twenty-thousand-gene biomarker that just re-says tumour grade isn't worth the assay.

Ask the room: Should age and tumour size be features, or should we test the transcriptome on its

own?

A *Genuine argument both ways. Including them tests "transcriptome + clinical" as a package; leaving them out isolates whether the genes add anything beyond what's already free. Which you pick depends on the claim you want to make.*

Slide 10 — Why this is hard: $p \gg n$

(~4 min)

Transcriptomics data contains a lot of information. It contains the expression of single genes, it contains the gene regulatory networks, it is a proxy for genetic mutations! It's a lot of information summarised in the expression of 20k genes. Which brings us to the single most important fact in this whole course: **p much bigger than n.**

- **p** = number of features $\approx$ twenty thousand genes.
- **n** = number of samples $\approx$ a few hundred patients.

In most of machine learning it's the other way round: millions of images, each with a modest number of features. That's the $n \gg p$ world, and nearly all your ML intuition was built there. In genomics it's flipped: twenty thousand features, two hundred patients. **Short and fat.**

Sit with this, because it explains most of what follows. And notice — it holds *even for a big cohort*. METABRIC has ~1,400 HR-positive patients, still tiny next to twenty thousand genes. We don't escape this regime by getting lucky with cohort size.

Ask the room: How many patients do you reckon you'd need before twenty thousand features stopped feeling dangerous? And is that number remotely realistic in oncology?

A *You'd want samples to comfortably outnumber features — tens of thousands of patients, at least. Not realistic: recruiting thousands of uniformly-treated, uniformly-profiled patients is rare and hugely expensive. That's the whole point — "just collect more data" usually isn't on the table, so the discipline has to come from method, not sample size.*

Slide 11 — The curse of dimensionality

(~5 min)

Let me give you the intuition for why $p \gg n$ is so dangerous.

Take some points:

- on a **1D** line, there is only one way to keep the points separated: which is left right, right?
- In **2D**, you have more possibilities to separate them
- in a **3D** cube, it becomes even easier to separate them;
- in **twenty thousand** dimensions, most of the space is empty so it's easy to find a combination of genes to separate 1000 points. This is not biology, it's geometry, it's just

noise. It happens very likely by chance, not by biology! This is why in $p \gg n$ condition, overfit is the default results, not an exception.

Myth to bury: "perfect separation on my data means I found real signal." No, because it might be overfitting

Slide 12 — Overfitting in the $p \gg n$ regime

(~4 min)

Overfitting = your model memorises *these specific patients* — instead of learning reproducible biology that carries over to new patients. In the $p \gg n$ regime, this is the default, for the model, it's easier to memorise the patients than learning biology that is reproducible.

The sign of overfitting: crazy performance on the training data, then basically a disaster on new data. Like on the plots here, training AUC of 0.99, dropping to 0.55, which is basically coin flipping, on an independent cohort.

Ask the room: What would actually convince you that a model learned biology, not noise?

A *Independent validation on a separate cohort; feature selection that stays stable when you resample; and biological plausibility of the genes. We'll build all three.*

Slide 13 — Signatures are not unique

(~4 min)

When you develop a predictive model, you will see that many solutions exist.

1. you can build **many different** gene sets — barely overlapping, sometimes not at all — that predict outcome basically equally well;
2. If you just **resample the patients** you will see that the specific genes that get picked change.

What does this mean?: a gene showing up in your signature is *weak* evidence it's biologically causal. It just tells you the gene co-varies with outcome in your particular sample; it doesn't mean the gene drives the disease.

Ask the room: how do we avoid randomness and pick at least weak evidence? External validation, gene set stability against bootstrapping, pathway analysis

Myth to retire: "the genes in a predictive signature are the biological drivers." Predictive $\neq$ causal $\neq$ stable.

Slide 14 — Batch effects

(~4 min)

Another confounder that you guys might be all familiar is: **batch effects**.

A batch effect comes from technical variation linked to *how, when, where and who* processed the samples. The problem isn't that it exists (it always does). The problem is evident when:

- it is **bigger** than the biological signal you care about, and
- — far worse — it can be **correlated with your outcome**.

Imagine all responders are from a given hospital and all no responder are from the other hospital, here one could argue that there is a problem with the second hospital (or not). But anyway, when you build your model, if you do not check the confounding, you would see a perfect model that you validate in your hold out test!! but not externally of course because you did not capture the biology, just the batch effect

Just to remember: standard normalisation does not necessarily correct for batch effect. You have to build a normalisation that explicitly correct for it!

Slide 15 — Seeing batch effects with PCA

(~4 min)

So how do we actually *see* a batch effect? With **PCA**

PCA squeezes that twenty-thousand-dimensional matrix onto a few axes that capture the most variation, so we can see the dominant source of variability. Here is what you do: take the **same** PCA plot and colour it two ways.

- First by **batch** (platform, in our case);
- then by the **biological label** (relapse vs not).

If the points split cleanly by batch but smear together by biology, you've found your confounder. *The array knew the platform before it knew the tumour.*

Crucially: PCA *reveals* the problem; it does **not** fix it. Correction methods like ComBat come later, must be done carefully — inside the training fold only, or you leak — and can actually scrub out real biology if batch and outcome are tangled.

Ask the room: Suppose PC1 perfectly separates two hospitals, and all your responders happen to be in one. Can you still correct for batch?

A *No. You can't statistically separate batch from outcome when they're perfectly collinear — they point the same direction. No algorithm can untangle that. It's a study-design failure, not something you fix in software.*

Slide 16 — Data leakage

(~3 min)

Now let's move to the single most expensive mistake in our field: **data leakage**.

The definition is simple: leakage is any time information that *shouldn't* be available at prediction

time sneaks into the model-building. This is effectively cheating. We have spectacular results in development phase on the hold out test, then falls apart in the real world.

From experience: **leakage is the number one reason biomarkers fail to replicate**. More than bad algorithms. More than small samples. It's the big one.

Today's goal is just to *recognise* it. Preventing it systematically is the workflow we'll build at the end of this lecture.

Data Leakage is almost never a coding bug, it's more conceptual error.

Slide 17 — Flavours of leakage

(~5 min)

The four types of leakage you'll meet in transcriptomics, each with an example:

1. **Feature-selection leakage** (the classic killer): you rank all twenty thousand genes by association with outcome — using *all* your samples — and *then* split. The genes you picked have already "seen" the test labels. with 20k genes and 100 patients, some genes correlate with outcome by pure chance; select them on all your data and your held-out test set is contaminated before you've trained a thing.
2. **Normalisation leakage**: you compute scaling stats (means, variances, quantile normalisation) across train + test together. The test set has leaked into preprocessing.
3. **Sample leakage**: technical replicates, two biopsies from one tumour, or related individuals landing on opposite sides of the split. The model has effectively already met the test patient.
4. **Temporal / label leakage**: using a feature only knowable *after* the outcome — e.g. a post-treatment measurement to "predict" treatment response.

It's really important that you guys keep this in mind. Especially because you have few samples and lots of features, feature selection leakage would explode

Slide 18 — Why biomarkers fail validation

(~4 min)

Let's now summarise the **anatomy of failure** — once you see the pattern you'll spot it everywhere. It's some mix of:

- optimistic bias (from leakage and from tuning against the test set);
- underpowered cohorts in the $p \gg n$ regime;
- batch confounding misread as biology;
- overfitting to a single dataset with no independent cohort;
- selective reporting — quietly trying many analyses, showing the best one.

The signature pattern is always the same: very successful paper with crazy AUC in the hold out test. This AUC drops a lot in independent validation. Our goal is really to build a model that might not

reach the 0.95 AUC or whatever the metrics is, but would not fail in independent validation

Slide 19 — What success looks like

(~3 min)

The clinically approved biomarkers, the one used in the clinics did not win because of the model. They are logistic regression. They will because of what is around the model, the process.

- signatures **pre-specified and locked** before validation;
- validated **independently**, across multiple separate cohorts;
- and they showed **clinical utility** — they actually changed treatment decisions and improved outcomes, not just a statistical association.

Ask the room: What's the difference between a biomarker that's *associated* with outcome and one that's *clinically useful*?

A *Useful means it changes a decision and improves outcomes compared to the tools we already have. Association alone does none of that — a marker can correlate beautifully and still change nothing about how you'd treat the patient.*

Myth: "statistical significance equals clinical usefulness." It absolutely does not.

Slide 20 — The defensible workflow

(~4 min)

To get there, to have a biomarker that would not fail in the validation phase, we can follow the following workflow

1. **Lock** the question and the label.
 2. **Split** the data *first* — before you do anything else.
 3. Every data-dependent decision — normalisation parameters, feature selection, hyperparameter tuning — happens **inside the training data only**.
 4. **Touch the test set exactly once**, right at the end, to report your result.
-

Slide 21 — Train / validation / test

(~4 min)

We talked about splitting, train, test, validation. Let's define them

- **Training set:** you fit the model on it. Straightforward.
- **Validation set:** you use it to make choices — which genes, which settings, which configuration. Because you use it *over and over*, it gets "burned" — it can't give an honest

performance estimate anymore, precisely because you've been optimising against it.

- **Test set:** your one-shot, real-world estimate. Use it **once**. Never to guide a decision — the moment you tune against it, it's quietly become a second validation set, and you've blown the one unbiased estimate you had.

In oncology our cohorts are often too small to carve off a fixed validation set, so cross-validation stands in (covered properly later). One warning even now: in $p \gg n$, cross-validation has to wrap *all* your data-dependent steps inside each fold. Feature selection sitting *outside* the CV loop leaks just as badly as a single bad split.

Ask the room: Why can't I just peek at the test set "once or twice" to sanity-check during development?

A *Because each peek leaks. After the first look your estimate is no longer unbiased — you've started, even subconsciously, steering choices toward what scored well. There is no free peek.*

Myth: that "validation" and "test" are two words for the same thing. They have distinct, non-interchangeable jobs.

Slide 22 — Splitting pitfalls specific to omics

(~4 min)

About splitting, important notes for our omics data.

1. **Split by patient, never by sample.** If one patient gave a core biopsy *and* a surgical specimen, and one lands in train while the other lands in test, the model has effectively already seen the test patient. Leakage.
 2. **Respect batch.** Never let your split accidentally line batch up with outcome — you'd build the confounding straight into your evaluation.
 3. **Stratify** to keep class balance. Responders may be a small minority, and a careless random split can leave one fold with almost none of them.
 4. **Gold standard, said plainly:** an *independent external cohort* — different institution, different platform — is worth more than any internal split you can dream up.
-

Slide 23 — The main message

(~2 min)

Bring it all home to one sentence, and let it land:

Building a trustworthy predictive biomarker matters more than picking a sophisticated algorithm.

The hard, valuable, distinctly scientific work in our field is the framing, the label, the data structure, the splits, and the validation. It is **not** the choice of classifier. A simple model on a clean, leak-free,

externally-validated pipeline beats a state-of-the-art model on a compromised one. Every single time.

I want this to be the lens you look through for every paper you read and every project you build for the rest of this course.

Ask the room: Having heard all of this — would you rather have a fancy model, or a clean pipeline? And why?

A *A clean pipeline, every time. A fancy model on a leaky pipeline gives you a confident wrong answer; a simple model on a clean one gives you something you can actually trust and replicate. Hold onto your answer — we'll test it.*

Slide 24 — Recap and bridge to the practical

(~2 min)

Let's close by ticking off where we started. We:

- described the clinical prediction problem;
- learned how transcriptomic data gets represented — the wide, short matrix, X and y ;
- met $p \gg n$ and the curse of dimensionality;
- learned to diagnose batch effects with PCA;
- learned to recognise data leakage and why biomarkers fail;
- and laid out a defensible train/validation/test split.

That's the full checklist from the top of the hour.

In the practical, you'll do all of this by hand: load a real transcriptomic matrix (METABRIC + GSE6532); sanity-check the clinical metadata; derive a clean binary relapse label from the survival fields; check the class balance; run and interpret PCA; hunt down the cross-platform batch effect we set up earlier; and build a patient-level, stratified, batch-aware split.

Then, in **Lecture 2**, with the problem finally well-posed, we'll meet our first models — and I think you'll be surprised how almost *anticlimactic* the modelling feels once the framing is done properly.

Leave them with this: **today, you learned to distrust good results**. That reflexive suspicion of a result that looks too good is the single most valuable thing a translational data scientist owns. Guard it.

Thanks, everyone. Short break, then the practical.